

Machine Learning

Complete Full-Syllabus Detailed Notes

PCIT301 — Machine Learning (PCC)

Exam + Viva Revision Edition

This document is organized unit-by-unit according to the official PCIT301 syllabus. It is deliberately written in full sentences and point form rather than compressed shortcuts, so it can be used for last-minute theory revision and viva preparation.

Important: the official syllabus is the controlling checklist. Topics appearing in the lecture material but not explicitly named in the theory syllabus are included in a clearly marked supplementary section so that they are not confused with compulsory syllabus headings.

How to Use These Notes

- Read the definition of a topic first. For viva questions, the first two or three points are usually enough to begin an answer.
- For algorithms, remember the purpose, the main idea, the sequence of steps, and one important advantage or limitation.
- For metrics and formulas, know what each term means and when the metric is useful.
- For comparison questions, learn the difference table rather than memorizing isolated definitions.
- The final quick-revision section collects formulas, distinctions, and high-frequency viva facts without replacing the detailed notes above.

Official syllabus checklist: Unit I Introduction to Machine Learning; Unit II Regression; Unit III Classification; Unit IV Clustering; Unit V Association and Dimensionality Reduction. The syllabus explicitly names the individual topics covered below.

UNIT I — INTRODUCTION TO MACHINE LEARNING

1. Definition of Machine Learning

- Machine Learning is a subset of Artificial Intelligence that enables systems to learn from experience or data and improve their performance without being explicitly programmed for every individual decision.
- Arthur Samuel coined the term Machine Learning in 1959.
- The central idea is to provide data and a learning procedure so that the system can discover useful patterns and use those patterns to make predictions or decisions.
- A machine-learning system generally learns a relationship between input data and an output, or discovers structure in data when an output is not provided.
- The quality and quantity of data have a major influence on the performance of a machine-learning algorithm.

2. Artificial Intelligence, Machine Learning, and Deep Learning

- Artificial Intelligence is the broader field concerned with creating systems capable of performing tasks associated with intelligent behavior.
- Machine Learning is a subset of Artificial Intelligence that focuses on learning patterns from data.
- Deep Learning is a subset of Machine Learning that uses neural networks with multiple layers to learn complex patterns.
- Examples from the lecture material include self-driving cars as an AI example, Amazon Alexa as an ML example, and handwriting recognition as a deep-learning example.
- The relationship can be remembered as: AI is the broadest area; ML is inside AI; DL is inside ML.

3. Traditional Programming Approach versus Machine Learning

Traditional programming	Machine learning
Human explicitly defines rules and logic	Model learns patterns from data
Rules are manually written	Parameters are learned from examples
Performance depends strongly on predefined rules	Performance can improve with suitable data and learning
Manual feature engineering is often required	Some ML systems can learn useful representations automatically

4. Real-Life Applications of Machine Learning

- Fraud detection: suspicious transactions can be identified from transaction patterns.
- Recommendation systems: customer behavior and previous interactions can be used to recommend products, songs, videos, or other content.
- Spam filtering: labeled emails can be used to classify new messages as spam or not spam.
- Sentiment analysis: text can be analyzed to identify sentiment or opinion.
- Social-media analysis: patterns in user activity and content can be analyzed.
- Customer-service chatbots: language and interaction data can support automated responses.
- The lecture example of an e-commerce company combines fraud detection with prediction of customer behavior to improve recommendations and customer experience.

5. Data in Machine Learning

- Data is the raw material from which a machine-learning model learns patterns.

- A dataset contains observations or records. Each observation may contain several variables or features.
- A feature is an input variable or measurable characteristic used by a model.
- A target, output, or label is the value the model is expected to predict in supervised learning.
- In supervised learning, training examples contain both input information and the corresponding known output.
- In unsupervised learning, the input data is supplied without explicit target labels and the algorithm searches for structure or patterns.
- Good-quality and sufficiently large data generally supports more reliable predictions and useful insights.

6. Types of Machine Learning

6.1 Supervised Learning

- Supervised learning uses labeled data.
- Both input and corresponding output are known during training.
- The model learns a mapping from inputs to outputs and then uses that learned mapping to predict outputs for new inputs.
- Two major supervised tasks are classification and regression.
- Examples include predicting crop yield, predicting a numerical quantity, and classifying emails as spam or not spam.
- Common supervised algorithms in the course include Linear Regression, Logistic Regression, Decision Trees, KNN, Naive Bayes, and SVM.

6.2 Unsupervised Learning

- Unsupervised learning works with unlabeled data.
- The algorithm searches for hidden patterns, structures, groups, or relationships without a supplied target label.
- Clustering is a major unsupervised-learning task.
- Examples include grouping customers or users by common characteristics, image segmentation, and detecting unusual patterns.

6.3 Semi-Supervised Learning

- Semi-supervised learning uses a small amount of labeled data together with a larger amount of unlabeled data.
- It lies between supervised and unsupervised learning.
- The unlabeled data can provide additional information and can help improve generalization.
- The lecture material gives Google Photos as an example where only some images may have explicit labels while visual information from many images can still be used.

6.4 Reinforcement Learning

- Reinforcement learning involves an agent interacting with an environment.
- The agent observes a state, performs an action, and receives feedback such as a reward or penalty.
- The objective is to learn actions that improve long-term performance or cumulative reward.
- The agent repeatedly interacts with the environment and learns from the consequences of its actions.
- Examples listed in the lecture material include recommendation/search systems, self-driving cars, autocorrect tools, and games involving bots.

7. Concept of Feature

- A feature is an individual input variable or characteristic describing an observation.

- For house-price prediction, examples of features can include area, number of rooms, location-related variables, and age of the property.
- Features are also called predictors, input variables, or independent variables in appropriate contexts.
- A target is different from a feature: the target is what the model is trying to predict.

8. Feature Construction

- Feature construction means creating useful new features from existing data or information.
- A constructed feature can represent information that was not directly available as one original column.
- For example, separate height and weight measurements can be combined into a derived quantity such as Body Mass Index.
- Good feature construction can make useful relationships easier for a model to learn.

9. Feature Selection

- Feature selection means choosing the most relevant existing features and removing irrelevant or redundant features.
- Its purpose is to simplify the model, reduce unnecessary dimensionality, and potentially improve generalization and interpretability.
- Feature selection does not necessarily create new variables; it selects a subset of the variables that already exist.
- It can reduce computational cost and can reduce the effect of noisy or irrelevant inputs.

10. Feature Transformation

- Feature transformation changes features into a more suitable representation while preserving useful information.
- Common transformations include scaling, normalization, logarithmic transformation, and other mathematical transformations.
- Standardization commonly transforms a feature so that it has mean approximately zero and standard deviation approximately one.
- Transformation is especially important for algorithms that are sensitive to feature scale, such as KNN and SVM.

11. Curse of Dimensionality

- The curse of dimensionality refers to problems that arise as the number of dimensions or features increases.
- As dimensionality grows, data can become increasingly sparse in the feature space.
- Distance-based methods can become less effective because the concept of closeness becomes harder to use effectively in high-dimensional spaces.
- More dimensions can also increase computational cost and the risk of overfitting.
- Feature selection and dimensionality-reduction techniques such as PCA can help address high-dimensional data.

12. Training Dataset versus Testing Dataset

- The training dataset is used to learn the model parameters.
- The testing dataset is kept separate and is used to estimate performance on unseen data after training.
- Using the test data to repeatedly tune the model can make the reported test performance overly optimistic.
- A good train-test separation helps evaluate whether a model generalizes beyond the observations used to train it.

13. Hold-Out Validation

- Hold-out validation divides the available dataset into separate subsets, commonly training and validation or training and testing subsets.
- The model is trained on one portion and evaluated on the held-out portion.
- It is simple and computationally inexpensive.
- Its estimate can depend strongly on the particular split, especially when the dataset is small.

14. K-Fold Cross-Validation

- K-fold cross-validation divides the data into k approximately equal folds.
- In each iteration, one fold is used for validation and the remaining $k-1$ folds are used for training.
- The process is repeated until every fold has served as the validation fold.
- The k validation results are combined, commonly by averaging them, to obtain an overall estimate.
- Cross-validation makes better use of limited data than a single hold-out split.

15. Leave-One-Out Cross-Validation (LOOCV)

- LOOCV is an extreme form of K-fold cross-validation where k equals the number of observations.
- In each iteration, exactly one observation is held out for validation and all remaining observations are used for training.
- The process is repeated once for every observation.
- LOOCV uses almost all observations for training in each iteration, but it can be computationally expensive for large datasets.

UNIT II — REGRESSION

1. What is Regression?

- Regression is a supervised learning task in which the target is a continuous numerical value.
- The model learns a relationship between one or more input variables and a numerical output.
- Examples include predicting house price, temperature, sales, crop yield, or demand.

2. Types of Regression

- Simple Linear Regression uses one independent variable to predict one continuous dependent variable.
- Multiple Linear Regression uses two or more independent variables to predict one continuous dependent variable.
- Polynomial Regression uses polynomial terms such as x^2 , x^3 , and higher powers to model curved relationships.
- Logistic Regression is named as regression but is primarily used for classification; it estimates class probabilities rather than directly predicting a continuous target.

3. Simple Linear Regression

- Simple Linear Regression models the relationship between one input variable X and one continuous target Y .
- The basic equation is: $y = b_0 + b_1x$.
- b_0 is the intercept. It is the predicted value of y when x is zero.
- b_1 is the slope or coefficient. It represents the expected change in y for a one-unit increase in x , according to the fitted linear model.
- The model learns the coefficient and intercept from training data, commonly by minimizing a loss such as the sum of squared errors.

Formula: $y = b_0 + b_1x$

4. Multiple Linear Regression

- Multiple Linear Regression predicts one continuous target using two or more input features.
- A general form is $y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$.
- Each coefficient represents the modeled contribution of its feature while the other included features are held constant in the linear equation.
- It is useful when several factors jointly influence the target.

Formula: $y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$

5. Polynomial Regression

- Polynomial Regression extends a regression model by including polynomial terms.
- A simple example is $y = b_0 + b_1x + b_2x^2$.
- Higher-degree terms such as x^3 can also be included.
- The model can therefore represent curved relationships between an input and a continuous target.
- A high polynomial degree can make the model excessively flexible and increase the risk of overfitting.

6. Ridge Regression

- Ridge Regression is a regularized form of linear regression.
- It adds an L2 penalty to the objective, based on the squared magnitudes of model coefficients.

- The penalty discourages very large coefficients and can help control overfitting.
- Ridge generally shrinks coefficients toward zero but does not normally make them exactly zero.
- Ridge can be useful when predictors are highly correlated.

Key association: Ridge → L2 → squared coefficient penalty.

7. Lasso Regression

- Lasso Regression is a regularized form of linear regression using an L1 penalty.
- The penalty is based on the absolute values of the coefficients.
- Lasso can shrink some coefficients exactly to zero.
- Because coefficients can become zero, Lasso can perform a form of feature selection.
- The regularization strength controls how strongly coefficients are penalized.

Key association: Lasso → L1 → absolute coefficient penalty → can produce zero coefficients.

8. Overfitting and Underfitting

- Overfitting occurs when a model learns the training data too closely, including noise or accidental patterns, and therefore performs poorly on unseen data.
- An overfit model commonly has very good training performance but noticeably worse testing or validation performance.
- Underfitting occurs when a model is too simple to capture the important structure in the data.
- An underfit model generally performs poorly on both training and unseen data.
- Regularization, pruning, suitable model complexity, and validation methods can help control overfitting.

9. Mean Squared Error (MSE)

- MSE is the average of squared prediction errors.
- Squaring makes all errors non-negative and gives larger errors greater influence.
- Lower MSE indicates smaller average squared error for the evaluated predictions.

Formula: $MSE = (1/n) \times \sum (y_i - \hat{y}_i)^2$

10. Root Mean Squared Error (RMSE)

- RMSE is the square root of MSE.
- Because the square root is taken, RMSE has the same units as the target variable.
- RMSE is more sensitive to large errors than MAE because it is derived from squared errors.

Formula: $RMSE = \sqrt{MSE}$

11. Mean Absolute Error (MAE)

- MAE is the average of the absolute prediction errors.
- It treats each absolute error linearly rather than squaring it.
- MAE is generally less sensitive to large individual errors than MSE or RMSE.

Formula: $MAE = (1/n) \times \sum |y_i - \hat{y}_i|$

12. R² — Coefficient of Determination

- R^2 measures how much of the variation in the target is explained by the regression model relative to a baseline model using the target mean.
- An R^2 value of 1 indicates a perfect fit to the observed target values under the standard definition.
- An R^2 value of 0 means the model is no better than the mean-prediction baseline in terms of the standard sum-of-squares comparison.
- R^2 can be negative on unseen data when the model performs worse than the baseline.
- R^2 is not the same as classification accuracy.

Formula: $R^2 = 1 - [\sum(y_i - \hat{y}_i)^2 / \sum(y_i - \bar{y})^2]$

13. Regression Metric Comparison

Metric	What it measures	Important property
MSE	Average squared error	Strongly penalizes large errors
RMSE	Square root of average squared error	Same units as target
MAE	Average absolute error	More robust to large errors than MSE/RMSE
R^2	Explained variation relative to mean baseline	Higher is generally better; interpretation differs from error metrics

UNIT III — CLASSIFICATION

1. What is Classification?

- Classification is a supervised learning task in which the target is a category or class.
- Binary classification has two classes, such as spam and not spam.
- Multiclass classification has more than two possible classes.
- The model learns from labeled examples and predicts a class for new observations.

2. Sigmoid Function

- The sigmoid function is used in Logistic Regression to map a real-valued linear score into a value between 0 and 1.
- This value can be interpreted as an estimated probability for the positive class under the binary logistic model.
- A decision threshold can then be used to convert the probability into a class prediction.
- A common threshold is 0.5, although the threshold can be changed depending on the application.

Formula: $\sigma(z) = 1 / (1 + e^{(-z)})$

3. Decision Tree

- A Decision Tree is a supervised learning model represented as a tree-like structure.
- An internal node contains a feature-based test or split.
- A branch represents an outcome of a split.
- A leaf node contains the final prediction.
- Decision Trees can be used for both classification and regression.
- For classification, splits can be selected using impurity measures such as entropy/information gain or Gini impurity.

4. Entropy

- Entropy measures impurity or uncertainty in a classification node.
- A perfectly pure node has entropy zero.
- A node containing a mixture of classes has greater uncertainty.
- Decision Tree algorithms can use entropy reduction to choose useful splits.

Formula: Entropy = $-\sum p_i \log_2(p_i)$

5. Information Gain

- Information Gain measures the reduction in entropy produced by a split.
- A split with higher information gain provides a larger reduction in impurity under the entropy criterion.
- The tree can choose the feature and split that gives the best information gain among candidate splits.

Concept: Information Gain = parent entropy – weighted child entropy.

6. Gini Impurity

- Gini impurity is another measure of class impurity used for Decision Tree splitting.
- It is based on class probabilities within a node.
- A pure node has Gini impurity zero.

- Lower impurity represents a purer node under the Gini criterion.

Formula: $Gini = 1 - \sum p_i^2$

7. Decision Tree Pruning

- Pruning controls the complexity of a Decision Tree.
- Pre-pruning stops or limits tree growth before the tree becomes excessively complex.
- Post-pruning removes unnecessary branches after a larger tree has been grown.
- Pruning can reduce overfitting and improve generalization.
- Common tree hyperparameters related to complexity include maximum depth, minimum samples required to split, and minimum samples per leaf.

8. Naive Bayes Classifier

- Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem.
- Its simplifying assumption is that features are conditionally independent given the class.
- The classifier combines prior class probabilities with feature likelihoods to estimate the probability of each class.
- The predicted class is commonly the class with the highest posterior probability.
- Naive Bayes can be efficient and is widely used for text-related classification tasks.

Bayes principle: $P(\text{Class} \mid \text{Features})$ is proportional to $P(\text{Features} \mid \text{Class}) \times P(\text{Class})$.

9. Support Vector Machine (SVM)

- Support Vector Machine is a supervised learning algorithm used mainly for classification and also adaptable to regression.
- The maximum-margin idea seeks a decision boundary that separates classes with the largest possible margin.
- Support vectors are training observations that are most important in defining the separating boundary.
- A larger margin is generally associated with better generalization under the SVM formulation.
- Kernel methods can allow SVMs to model nonlinear decision boundaries by implicitly working in a transformed feature space.

10. Confusion Matrix

- A confusion matrix summarizes classification predictions by comparing actual classes with predicted classes.
- True Positive (TP): actual positive and predicted positive.
- True Negative (TN): actual negative and predicted negative.
- False Positive (FP): actual negative but predicted positive.
- False Negative (FN): actual positive but predicted negative.

11. Accuracy

- Accuracy is the proportion of all predictions that are correct.
- It counts both true positives and true negatives as correct predictions.
- Accuracy can be misleading for strongly imbalanced datasets because a model may achieve high accuracy while performing poorly on a minority class.

Formula: $\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$

12. Precision

- Precision measures how many observations predicted as positive are actually positive.
- It is especially important when false positives are costly.

Formula: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

13. Recall / Sensitivity

- Recall, also called sensitivity or true positive rate, measures how many actual positive cases are correctly identified.
- Recall is especially important when missing a positive case is costly.

Formula: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

14. F-Measure / F1-Score

- F1-score combines precision and recall using their harmonic mean.
- It is useful when both precision and recall matter and especially when class distribution is not balanced.

Formula: $\text{F1} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

15. Specificity

- Specificity is the true negative rate.
- It measures the proportion of actual negative cases correctly identified as negative.
- It is useful when avoiding false positives is important.

Formula: $\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$

16. ROC Curve

- A Receiver Operating Characteristic (ROC) curve shows the relationship between true positive rate and false positive rate across classification thresholds.
- The vertical axis is commonly the true positive rate, while the horizontal axis is the false positive rate.
- Changing the classification threshold changes the balance between detecting positives and generating false positives.

17. AUC

- AUC means Area Under the ROC Curve.
- It summarizes the ROC curve into a single measure of discrimination across thresholds.
- A larger AUC generally indicates better ability to distinguish positive and negative classes.

18. Ensemble Learning

- Ensemble learning combines multiple models to produce a stronger overall predictor.
- A weak learner is a model that performs only modestly better than a simple baseline; an ensemble can combine many such learners.
- Ensembles can be constructed in parallel or sequentially.
- Major ensemble ideas covered in the course are bagging, boosting, and Random Forest; stacking is a useful related ensemble technique from the lecture material.

19. Bagging

- Bagging stands for Bootstrap Aggregating.
- Multiple models are trained independently, usually on different bootstrap samples of the training data.

- Their predictions are combined, commonly by voting for classification or averaging for regression.
- Bagging mainly helps reduce variance and can improve stability.

20. Boosting

- Boosting builds an ensemble sequentially.
- Each new learner is trained with attention to the weaknesses or errors of the current ensemble.
- The final prediction combines the contributions of the sequential learners.
- Boosting can reduce bias and create a strong model from weak learners, although excessive complexity can still lead to overfitting.

21. AdaBoost

- AdaBoost is a boosting method that changes the importance or weight of training observations during sequential learning.
- Misclassified observations receive greater emphasis so that subsequent weak learners focus more strongly on difficult examples.
- The learners are combined into a weighted ensemble.

22. Gradient Boosting

- Gradient Boosting builds learners sequentially so that each new learner helps reduce the remaining error of the ensemble.
- In regression, the new learner can be understood as modeling residual errors.
- The method uses an optimization perspective based on a loss function.
- The final model is an additive combination of the sequential learners.

23. Random Forest

- Random Forest is an ensemble of Decision Trees.
- Each tree is commonly trained using a bootstrap sample of observations.
- At tree splits, random subsets of features are considered, increasing diversity among the trees.
- For classification, tree predictions are commonly combined by voting.
- For regression, predictions are commonly combined by averaging.
- The diversity of trees helps reduce variance and improve generalization.

24. Stacking — Supplementary Lecture Concept

- Stacking combines predictions from multiple base learners using another model called a meta-model or meta-learner.
- The base models generate predictions, and the meta-model learns how to combine those predictions.
- This is different from bagging, where models are independently trained and their predictions are directly aggregated.

UNIT IV — CLUSTERING

1. What is Clustering?

- Clustering is an unsupervised learning technique that groups observations according to similarity or structure in the data.
- Unlike supervised classification, clustering does not require predefined class labels for training.
- The aim is to create groups in which observations within the same cluster are relatively similar and observations in different clusters are relatively dissimilar.

2. Distance-Based Clustering

- Distance-based clustering uses a distance measure to determine how close or far observations are.
- K-Means and K-Medoids are important examples where distances influence assignment to cluster representatives.
- The chosen distance measure can affect the resulting clusters.

3. Density-Based Clustering

- Density-based clustering identifies groups based on regions containing many closely packed observations.
- Sparse regions can separate clusters.
- Density-based approaches can identify noise or isolated observations.
- DBSCAN is a common example of a density-based method and is included here as a representative of the density-based clustering category.

4. Hierarchical Clustering

- Hierarchical clustering builds a hierarchy of clusters.
- Agglomerative hierarchical clustering is bottom-up: it begins with each observation as its own cluster and repeatedly merges clusters.
- Divisive hierarchical clustering is top-down: it begins with one cluster and repeatedly splits it.
- A dendrogram is a tree-like diagram used to visualize the hierarchy and the sequence of merges or splits.

5. Euclidean Distance

- Euclidean distance is the straight-line distance between two points in Euclidean space.
- For two-dimensional points, it is the square root of the sum of squared coordinate differences.

Formula for two dimensions: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

6. Manhattan Distance

- Manhattan distance is the sum of the absolute coordinate differences.
- It can be visualized as movement along a grid rather than straight-line movement.

Formula: $d = \sum |x_i - y_i|$

7. Hamming Distance

- Hamming distance counts positions at which corresponding values differ.
- It is especially useful for comparing categorical, binary, or discrete sequences of equal length.

8. Minkowski Distance

- Minkowski distance is a generalized distance family controlled by a parameter p .

- With $p = 1$, it corresponds to Manhattan distance.
- With $p = 2$, it corresponds to Euclidean distance.

$$\text{Formula: } d(x,y) = [\sum |x_i - y_i|^p]^{1/p}$$

9. K-Means Clustering

- K-Means partitions observations into K clusters.
- K must be specified before the algorithm runs.
- Each cluster is represented by a centroid, which is the mean position of observations assigned to that cluster.
- The algorithm alternates between assigning observations to their nearest centroid and updating each centroid using the mean of its assigned observations.
- The process continues until assignments or centroids stabilize according to the stopping condition.
- K-Means is sensitive to initialization and can be influenced by outliers because centroids are means.

10. K-Means Algorithm Steps

1. Choose the number of clusters K.
2. Initialize K centroids.
3. Assign each observation to the nearest centroid according to the chosen distance measure.
4. Recalculate each centroid as the mean of observations assigned to that cluster.
5. Repeat assignment and centroid-update steps until convergence or the stopping criterion is reached.

11. Elbow Method

- The Elbow Method helps select a suitable K for K-Means.
- For several candidate K values, a clustering cost such as within-cluster sum of squares (WCSS) is computed.
- The cost generally decreases as K increases.
- The elbow is the point after which adding more clusters produces relatively smaller improvement.

12. Silhouette Score — Supplementary Lecture Concept

- Silhouette Score evaluates how well observations fit their assigned clusters compared with neighboring clusters.
- It considers both within-cluster cohesion and separation from other clusters.
- A higher silhouette score generally indicates better-defined clustering.
- Unlike the Rand Index, it does not require ground-truth class assignments.

13. K-Medoids

- K-Medoids is a partitioning clustering method similar in purpose to K-Means.
- Instead of using a calculated mean centroid, each cluster is represented by a medoid.
- A medoid is an actual observation from the dataset.
- Because the representative is an actual data point rather than a mean, K-Medoids is generally more robust to extreme observations than K-Means.
- K-Medoids still requires the number of clusters K to be specified.

14. K-Medoids Algorithm — General Steps

1. Choose the number of clusters K and select initial medoids.

- 2. Assign each observation to the nearest medoid.
- 3. Evaluate possible replacements of medoids with non-medoid observations.
- 4. Choose replacements that improve the clustering objective.
- 5. Repeat assignment and medoid-update steps until no useful improvement remains or the stopping condition is reached.

15. Rand Index

- Rand Index evaluates the agreement between a clustering assignment and known ground-truth assignments.
- It considers pairs of observations and checks whether the predicted clustering agrees with the reference relationship.
- A higher Rand Index indicates greater agreement with the reference clustering.
- It is different from the Silhouette Score, which evaluates cluster structure without requiring ground-truth labels.

16. K-Nearest Neighbour (KNN)

- KNN is a supervised learning algorithm that predicts using the K nearest training observations.
- For classification, the predicted class is commonly selected by majority vote among the nearest neighbors.
- For regression, the predicted numerical value can be calculated from the neighbors, commonly using their average.
- The choice of K is a hyperparameter.
- A very small K can make predictions sensitive to noise and can increase overfitting.
- A very large K can make predictions smoother and can increase underfitting.
- Feature scaling is important because distance calculations can be dominated by features with larger numerical scales.

UNIT V — ASSOCIATION AND DIMENSIONALITY REDUCTION

1. Association Rule Learning

- Association rule learning discovers relationships among items or events that frequently occur together.
- A rule is commonly written as $A \rightarrow B$, meaning that the presence of A is associated with the presence of B.
- The method is especially common in transaction or market-basket analysis.

2. Market Basket Analysis

- Market Basket Analysis studies customer transaction records to discover products that tend to occur together.
- For example, if customers who purchase item A often also purchase item B, the rule $A \rightarrow B$ can be investigated.
- The purpose is to discover useful associations rather than to establish causation.

3. Apriori Algorithm

- Apriori is an association-rule mining algorithm used to find frequent itemsets and then derive association rules.
- It relies on the frequent-itemset property: if an itemset is frequent, its subsets must also be frequent.
- Conversely, if an itemset is known to be infrequent, larger itemsets containing it can be pruned from consideration.
- Apriori therefore reduces the number of candidate itemsets that need to be considered.

4. General Apriori Procedure

- 1. Start with individual items and calculate their support.
- 2. Keep the itemsets whose support satisfies the chosen minimum-support threshold.
- 3. Generate larger candidate itemsets from the frequent itemsets.
- 4. Prune candidates containing infrequent subsets.
- 5. Calculate support for the remaining candidates and repeat until no further frequent itemsets can be generated.
- 6. Generate association rules from frequent itemsets and evaluate the rules using measures such as confidence and lift.

5. Support

- Support measures how frequently an itemset occurs in the entire transaction dataset.
- It is useful for identifying itemsets that occur often enough to be considered frequent.

For itemset A: $\text{Support}(A) = \text{number of transactions containing A} / \text{total number of transactions}$.

6. Confidence

- For a rule $A \rightarrow B$, confidence measures how often B occurs among transactions that contain A.
- It is a conditional measure: it focuses on transactions where the antecedent A is present.

$\text{Confidence}(A \rightarrow B) = \text{Support}(A \cup B) / \text{Support}(A)$

7. Lift

- Lift compares the observed co-occurrence of A and B with the co-occurrence expected if they were independent.
- Lift greater than 1 generally indicates a positive association.
- Lift equal to 1 corresponds to the independence baseline.

- Lift less than 1 generally indicates a negative association relative to the independence expectation.

$$\text{Lift}(A \rightarrow B) = \text{Confidence}(A \rightarrow B) / \text{Support}(B)$$

8. PCA — Principal Component Analysis

- Principal Component Analysis is a dimensionality-reduction technique.
- PCA transforms the original features into a new set of variables called principal components.
- The first principal component captures the greatest variance possible in its direction.
- Subsequent principal components capture the maximum remaining variance subject to being orthogonal to the preceding components.
- By retaining only the first few components, dimensionality can be reduced while preserving a large portion of the variation in the data.
- PCA is generally an unsupervised transformation because it does not require class labels.

9. Why Dimensionality Reduction is Used

- It reduces the number of features used to represent data.
- It can reduce computational cost.
- It can help visualize high-dimensional data in fewer dimensions.
- It can reduce redundancy and help address some effects of the curse of dimensionality.
- It can sometimes reduce noise and improve downstream model performance, although this is not guaranteed.

10. PCA — Standardization

- PCA is variance-based, so features measured on very different scales can cause large-scale variables to dominate the variance.
- Standardization is therefore commonly performed before PCA when feature scales are substantially different.
- A common standardized value is $z = (x - \text{mean}) / \text{standard deviation}$.

11. PCA — Covariance Matrix

- After suitable preprocessing, PCA can analyze relationships among features through the covariance matrix.
- The covariance matrix summarizes how pairs of features vary together.
- The eigenvectors of the covariance matrix provide principal directions.
- The corresponding eigenvalues indicate how much variance is associated with those directions.

12. PCA — Eigenvectors

- An eigenvector of the covariance matrix gives a principal direction in feature space.
- The direction of an eigenvector describes a direction along which PCA examines variation.
- Principal component directions are mutually orthogonal in standard PCA.

13. PCA — Eigenvalues

- An eigenvalue associated with a principal component indicates the amount of variance captured by that component.
- Larger eigenvalues correspond to components that explain more variance.
- Principal components are commonly ordered from largest eigenvalue to smallest eigenvalue.

14. PCA — Orthogonality

- Orthogonality means that the principal component directions are perpendicular to each other under the standard Euclidean inner-product interpretation.
- This prevents the selected principal directions from representing the same direction repeatedly.
- Each subsequent component therefore captures a new direction of variation.

15. PLS — Partial Least Squares Subset Selection / Dimensionality Reduction

- Partial Least Squares constructs latent components from predictor variables while considering their relationship with the response variable.
- Unlike PCA, which is primarily concerned with explaining variance in the predictors, PLS uses information about the response while constructing components.
- PLS can be useful when there are many correlated predictors and the objective is prediction or explanation of a response.
- The syllabus lists Partial Least Squares Subset Selection under dimensionality reduction and feature reduction.

16. Feature Reduction versus Feature Transformation

- Feature reduction can mean selecting fewer existing features or constructing a lower-dimensional representation.
- Feature selection keeps a subset of the original variables.
- Feature extraction or transformation creates new variables from the original variables.
- PCA is an example of feature transformation/extraction because its principal components are new combinations of the original features.

SUPPLEMENTARY LECTURE TOPICS

These topics appeared in the uploaded lecture material reviewed during preparation but are not all explicitly named as individual headings in the official PCIT301 theory syllabus. They are included so that the notes do not silently omit useful lecture content.

1. Multiclass Classification

- Multiclass classification has more than two possible classes.
- A single observation is generally assigned to one class among the available classes.
- Common supervised algorithms such as Decision Trees, Naive Bayes, KNN, and SVM can be adapted to multiclass settings.

2. Multi-Label Classification

- In multi-label classification, one observation can have multiple labels at the same time.
- This differs from multiclass classification, where one observation is generally assigned one class.
- Strategies discussed in the lecture material include Binary Relevance, Classifier Chains, Label Powerset, and ensemble methods.

3. Imbalanced Data

- Class imbalance occurs when one class has substantially more observations than another.
- Accuracy can be misleading under strong imbalance.
- Oversampling increases the representation of minority examples.
- Undersampling reduces the representation of majority examples.
- Precision-recall analysis can be particularly informative when the positive or minority class is important.
- Ensemble methods designed for imbalance can also be used in appropriate settings.

4. Gradient Boosting Family

- Gradient Boosting is a general sequential boosting framework based on minimizing a loss function.
- XGBoost is an optimized and regularized implementation of gradient boosting with additional engineering and algorithmic improvements.
- CatBoost is another gradient-boosting implementation designed to work especially well with categorical features.

5. Voting Ensembles

- Hard voting combines predicted class labels and chooses the class receiving the strongest vote.
- Soft voting combines predicted class probabilities and chooses the class with the strongest combined probability according to the ensemble rule.
- Weighted voting can give different models different influence.

6. Averaging and OOB Concepts

- For regression ensembles, averaging predictions is a common aggregation strategy.
- Out-of-bag evaluation uses observations that were not selected for a particular bootstrap sample to estimate that tree's performance within a Random Forest.
- Bootstrap sampling contributes diversity to bagging-based ensembles.

7. DBSCAN Concepts

- DBSCAN is a density-based clustering method.
- Epsilon (ϵ) defines the neighborhood radius.
- MinPts specifies the minimum number of points needed to characterize a dense neighborhood under the algorithm's definitions.
- DBSCAN can identify dense clusters without requiring K to be specified in advance.
- Points that do not belong to dense regions can be marked as noise.

8. Silhouette Score

- Silhouette analysis compares how close an observation is to its own cluster with how close it is to neighboring clusters.
- Higher values generally indicate better separation and cohesion.
- It is an internal clustering evaluation measure and does not require ground-truth labels.

9. Recommendation Systems — Outside the Official PCIT301 Theory Syllabus

- The uploaded lecture material also covered recommendation systems.
- Collaborative filtering uses user-item interaction patterns.
- Content-based filtering uses characteristics of items and user preferences.
- Hybrid systems combine multiple recommendation approaches.
- Cold-start refers to difficulty making recommendations for new users or new items with little interaction data.
- Explicit feedback includes direct ratings or similar feedback; implicit feedback includes actions such as views, clicks, or purchases.
- Because recommendation systems are not explicitly listed in the official PCIT301 theory syllabus, treat this section as supplementary rather than as a compulsory syllabus checklist.

HIGH-VALUE VIVA COMPARISONS

Supervised vs Unsupervised

Supervised	Unsupervised
Uses labeled data	Uses unlabeled data
Predicts known target	Discovers structure/patterns
Classification and regression	Clustering is a major task

Ridge vs Lasso

Ridge	Lasso
L2 regularization	L1 regularization
Squared coefficient penalty	Absolute coefficient penalty
Shrinks coefficients	Can make coefficients exactly zero

Precision vs Recall

Precision	Recall
Correctness of predicted positives	Coverage of actual positives
$TP/(TP+FP)$	$TP/(TP+FN)$
Sensitive to false positives	Sensitive to false negatives

Bagging vs Boosting

Bagging	Boosting
Models trained independently	Models trained sequentially
Bootstrap samples are commonly used	Later learners focus on remaining errors
Mainly reduces variance	Can reduce bias and build a strong learner

K-Means vs K-Medoids

K-Means	K-Medoids
Uses calculated mean centroid	Uses actual data-point medoid
More affected by outliers	Generally more robust to extreme observations
Requires K	Requires K

Rand Index vs Silhouette

Rand Index	Silhouette Score
Can use ground-truth assignments	Does not require ground truth
Measures agreement with reference clustering	Measures cohesion and separation

Feature Selection vs PCA

Feature selection	PCA
Keeps selected original features	Creates transformed principal components
Removes features	Transforms feature space
Improves simplicity/interpretability	Reduces dimensions while retaining variance

Support vs Confidence vs Lift

Support	Confidence	Lift
Overall itemset frequency	B given A	Observed vs expected association
Entire dataset	Conditional on antecedent	Relative to independence baseline

FORMULA AND FACT SHEET

Simple Linear Regression: $y = b_0 + b_1x$

Multiple Linear Regression: $y = b_0 + b_1x_1 + \dots + b_nx_n$

Sigmoid: $\sigma(z) = 1 / (1 + e^{(-z)})$

MSE = $(1/n) \times \sum (y_i - \hat{y}_i)^2$

RMSE = $\sqrt{\text{MSE}}$

MAE = $(1/n) \times \sum |y_i - \hat{y}_i|$

$R^2 = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$

Entropy = $-\sum p_i \log_2(p_i)$

Gini impurity = $1 - \sum p_i^2$

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

Precision = $TP / (TP + FP)$

Recall/Sensitivity = $TP / (TP + FN)$

Specificity = $TN / (TN + FP)$

$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

Euclidean distance = $\sqrt{[\sum (x_i - y_i)^2]}$

Manhattan distance = $\sum |x_i - y_i|$

Minkowski distance = $[\sum |x_i - y_i|^p]^{1/p}$

Support(A) = transactions containing A / total transactions

Confidence(A $\rightarrow$ B) = Support(A $\cup$ B) / Support(A)

Lift(A $\rightarrow$ B) = Confidence(A $\rightarrow$ B) / Support(B)

Standardization: $z = (x - \text{mean}) / \text{standard deviation}$

Fast Facts to Memorize

- Ridge $\rightarrow$ L2.
- Lasso $\rightarrow$ L1 and can set coefficients to zero.
- Recall = sensitivity = true positive rate.
- Specificity = true negative rate.
- Precision asks: among predicted positives, how many are actually positive?
- Recall asks: among actual positives, how many were detected?
- ROC plots TPR against FPR.
- AUC is the area under the ROC curve.
- Bagging is parallel/independent; boosting is sequential.
- AdaBoost emphasizes misclassified observations.
- Gradient Boosting adds learners that reduce remaining loss/error.
- Random Forest combines many randomized Decision Trees.
- K-Means uses a mean centroid; K-Medoids uses an actual data point.

- Agglomerative hierarchical clustering is bottom-up.
- Divisive hierarchical clustering is top-down.
- Minkowski with $p=1$ gives Manhattan; $p=2$ gives Euclidean.
- Apriori finds frequent itemsets and uses pruning based on the frequent-itemset property.
- Lift = 1 is the independence baseline.
- PCA creates principal components; eigenvectors give directions and eigenvalues indicate captured variance.
- Principal component directions are orthogonal in standard PCA.
- PLS considers the response while constructing latent components.

FINAL VIVA CHECKLIST

Unit I

- Define Machine Learning.
- Differentiate AI, ML, and DL.
- Differentiate traditional programming and ML.
- Give real-life ML applications.
- Explain supervised, unsupervised, semi-supervised, and reinforcement learning.
- Define feature, feature construction, feature selection, and feature transformation.
- Explain curse of dimensionality.
- Differentiate training and testing datasets.
- Explain hold-out, K-fold cross-validation, and LOOCV.

Unit II

- Define regression.
- Explain simple and multiple linear regression.
- Explain polynomial regression.
- Explain Logistic Regression and why sigmoid is used.
- Differentiate Ridge and Lasso.
- Define overfitting and underfitting.
- Explain MSE, RMSE, MAE, and R^2 .

Unit III

- Define classification and binary/multiclass classification.
- Explain Decision Tree structure.
- Define entropy, information gain, and Gini impurity.
- Explain pruning.
- Explain Naive Bayes and its conditional-independence assumption.
- Explain SVM and maximum margin.
- Define TP, TN, FP, FN.
- Explain accuracy, precision, recall, specificity, F1, ROC, and AUC.
- Differentiate bagging and boosting.
- Explain AdaBoost, Gradient Boosting, and Random Forest.

Unit IV

- Define clustering.
- Explain distance-based, density-based, and hierarchical clustering.
- Define Euclidean, Manhattan, Hamming, and Minkowski distances.
- Explain K-Means and its algorithm.
- Explain centroid and elbow method.

- Explain K-Medoids and medoid.
- Differentiate K-Means and K-Medoids.
- Explain Rand Index.
- Explain KNN and the effect of K.

Unit V

- Define association rules and Market Basket Analysis.
- Explain Apriori.
- Define support, confidence, and lift.
- Explain dimensionality reduction.
- Explain PCA.
- Explain eigenvalues and eigenvectors in PCA.
- Explain orthogonality of principal components.
- Differentiate feature selection and PCA.
- Explain PLS and how it differs conceptually from PCA.

This document is intended as a complete study/revision aid. For the exam, prioritize the official syllabus headings first and use the supplementary lecture sections for additional preparation.